log messages written inside the scope block will have the associated data, whereas those outside won't.

Listing 26.8 Adding scope properties to log messages with `BeginScope`

```
_logger.LogInformation("No, I don't have scope");          ❶
using(_logger.BeginScope("Scope value"))                   ❷
using(_logger.BeginScope(new Dictionary<string, object>    ❸
    {{ "CustomValue1", 12345 } }))                         ❸
{
    _logger.LogInformation("Yes, I have the scope!");      ❹
}
_logger.LogInformation("No, I lost it again");             ❶
```

❶ Log messages written outside the scope block don't include the scope state.

❷ Calling BeginScope starts a scope block, with a scope state of "Scope value".

❸ You can pass anything as the state for a scope.

❹ Log messages written inside the scope block include the scope state.

The scope state can be any object at all: an `int`, a `string`, or a `Dictionary`, for example. It's up to each logging provider implementation to decide how to handle the state you provide in the `BeginScope` call, but typically, it is serialized using `ToString()`.

TIP The most common use for scopes I've found is to attach additional key-value pairs to logs. To achieve this behavior in Seq, you need to pass `Dictionary <string, object>` as the state object. Nicholas Blumhardt, the creator of Serilog and Seq, has examples and the reasoning for this on his blog in the "The semantics of ILogger.BeginScope()" article: http://mng.bz/GxDD.

When the log messages inside the scope block are written, the scope state is captured and written as part of the log, as shown in figure 26.13. The `Dictionary<>` of key-value pairs is added directly to the log message